



RoomDatabase

Ha Noi Aug - 2023

OPEN MIND - OPEN DOORS



Table of content

OPEN MIND - OPEN DOORS

Overview

3

Define database using Entities

7

Access data using DAOs

11

Define relationships between objects

15

Prepopulate your database

23

Migrate your Room database

28

Test database

31

Migrate from SQLite to Room

34

Save data using SQLite

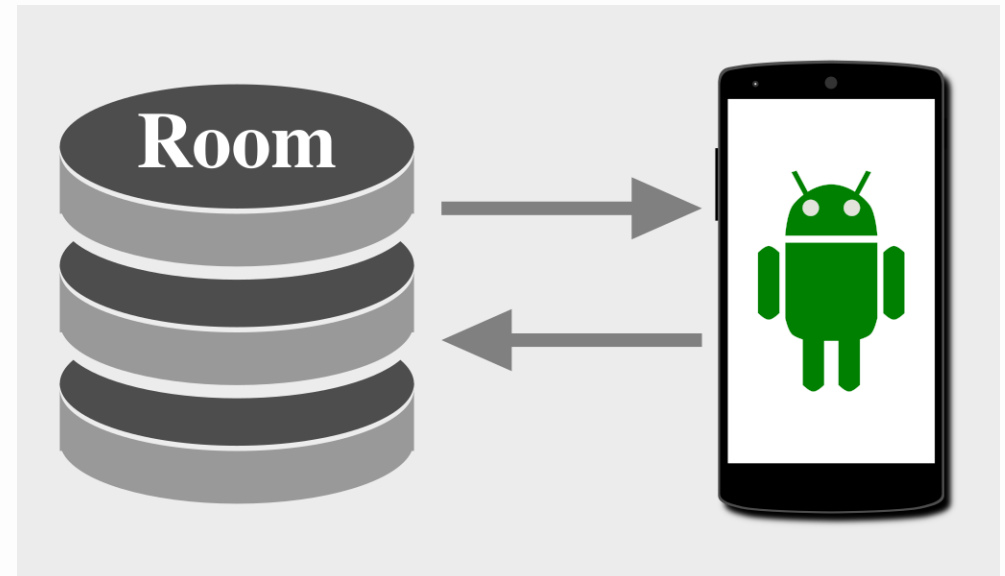
37

Overview



What's Room Database ?

- » **The Room** persistence library provides an abstraction layer over **SQLite** to allow fluent database access while harnessing the full power of **SQLite**.
- » In particular, **Room** provides the following benefits:
 - » Compile-time verification of SQL queries.
 - » Convenience annotations that minimize repetitive and error-prone boilerplate code.
 - » Streamlined database migration paths.



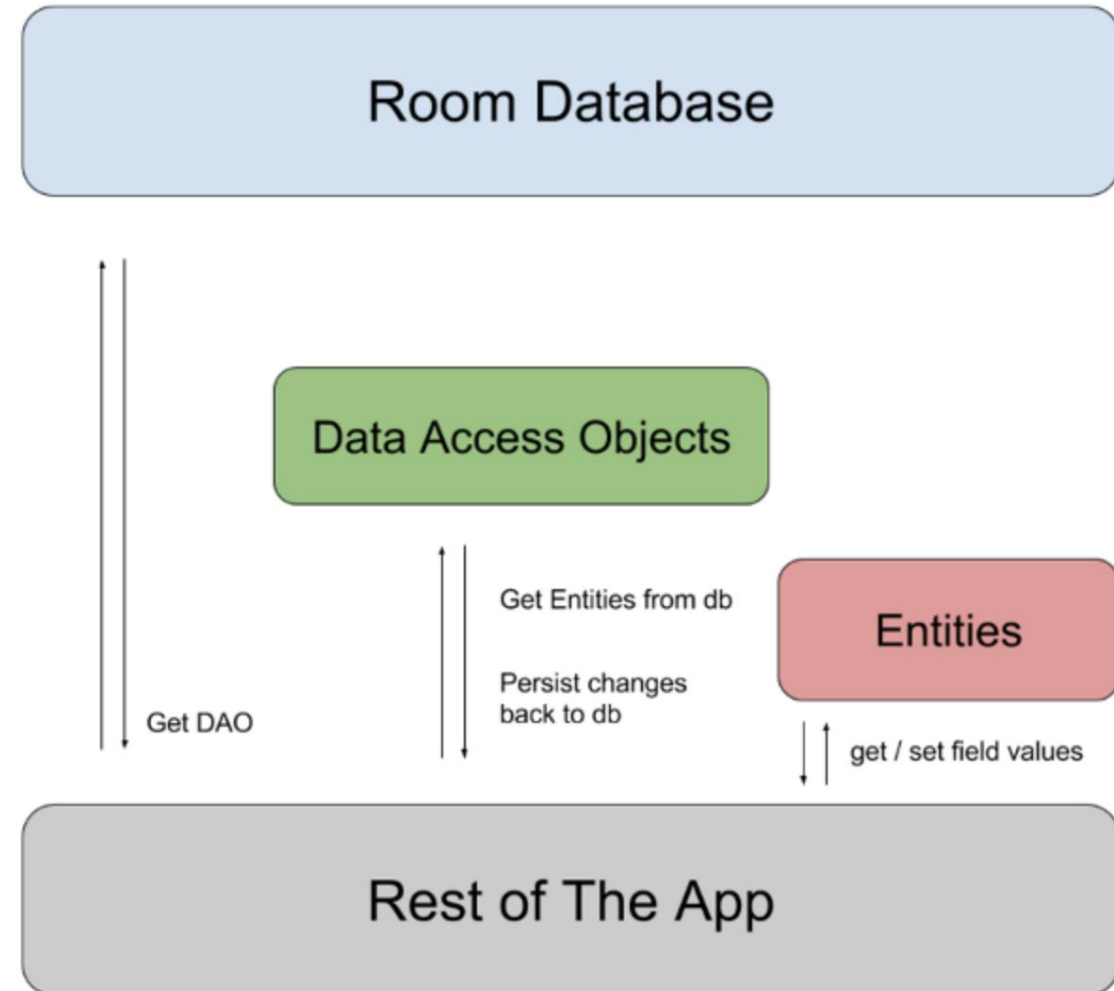
Set up

» To use Room in your app, add the following dependencies to your app's build.gradle file:

```
dependencies {  
    val room_version = "2.5.2"  
  
    implementation("androidx.room:room-runtime:$room_version")  
    annotationProcessor("androidx.room:room-compiler:$room_version")  
  
    // To use Kotlin annotation processing tool (kapt)  
    kapt("androidx.room:room-compiler:$room_version")  
    // To use Kotlin Symbol Processing (KSP)  
    ksp("androidx.room:room-compiler:$room_version")  
  
    // optional - Kotlin Extensions and Coroutines support for Room  
    implementation("androidx.room:room-ktx:$room_version")  
  
    // optional - RxJava2 support for Room  
    implementation("androidx.room:room-rxjava2:$room_version")  
  
    // optional - RxJava3 support for Room  
    implementation("androidx.room:room-rxjava3:$room_version")  
  
    // optional - Guava support for Room, including Optional and ListenableFuture  
    implementation("androidx.room:room-guava:$room_version")  
  
    // optional - Test helpers  
    testImplementation("androidx.room:room-testing:$room_version")  
  
    // optional - Paging 3 Integration  
    implementation("androidx.room:room-paging:$room_version")  
}
```

Primary components

- The [database class](#) that holds the database and serves as the main access point for the underlying connection to your app's persisted data.
- [Data entities](#) that represent tables in your app's database.
- [Data access objects \(DAOs\)](#) that provide methods that your app can use to query, update, insert, and delete data in the database.

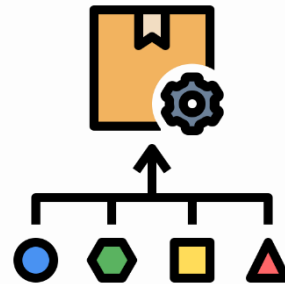


Define data using Entities



Anatomy of an entity

- » You define each Room entity as a class annotated with [@Entity](#). A Room entity includes fields for each column, including one or more columns that make up the [primary key](#).
- » By default, Room uses the class name as the database table name. If you want the table to have a different name, set the [tableName](#) property of the [@Entity](#) annotation.
- » If you want a column to have a different name, add the [@ColumnInfo](#) annotation to the field and set the [name](#) property.



Anatomy of an entity

Kotlin

Java

```
@Entity(tableName = "users")
data class User (
    @PrimaryKey val id: Int,
    @ColumnInfo(name = "first_name") val firstName: String?,
    @ColumnInfo(name = "last_name") val lastName: String?
)
```



Define a primary key

- » Each Room entity must define a primary key that uniquely identifies each row. The most straightforward way of doing this is to annotate a single column with @PrimaryKey.

Kotlin

Java

```
@PrimaryKey val id: Int
```



Access data using DAOs



Anatomy of a DAO

- » You can define each DAO as either an interface or an abstract class.
- » You must always annotate your DAOs with `@Dao`, they do define one or more methods for interacting with the data in your app's database.

Kotlin

Java

```
@Dao
interface UserDao {
    @Insert
    fun insertAll(vararg users: User)

    @Delete
    fun delete(user: User)

    @Query("SELECT * FROM user")
    fun getAll(): List<User>
}
```



Convenience methods

- » Room provides convenience annotations for defining methods that perform simple insertions, updates, and deletions without requiring you to write a SQL statement.

```
@Dao
interface UserDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    fun insertUsers(vararg users: User)

    @Insert
    fun insertBothUsers(user1: User, user2: User)

    @Insert
    fun insertUsersAndFriends(user: User, friends: List<User>)
}
```

```
@Dao
interface UserDao {
    @Update
    fun updateUsers(vararg users: User)
}
```

```
@Dao
interface UserDao {
    @Delete
    fun deleteUsers(vararg users: User)
}
```

Query methods

- » The [@Query](#) annotation lets you write SQL statements and expose them as DAO methods.
- » Use these query methods to query data from your app's database or when you need to perform more complex insertions, updates, and deletions.
- » Room validates SQL queries at compile time.

Kotlin

Java

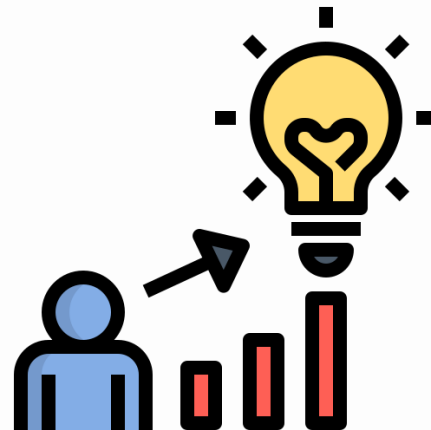
```
@Query("SELECT * FROM user")  
fun loadAllUsers(): Array<User>
```


Define relationships between objects



Two possible approaches

- » In **Room**, there are two ways to define and query a relationship between entities:
- » Using an intermediate data class with embedded objects.
- » Using a relational query method with a multimap return type.



Intermediate data class

- » In the intermediate data class approach, you define a data class that models the relationship between your Room entities.
- » This data class holds the pairings between instances of one entity and instances of another entity as [embedded objects](#).

```
@Dao
interface UserBookDao {
    @Query(
        "SELECT user.name AS userName, book.name AS bookName " +
        "FROM user, book " +
        "WHERE user.id = book.user_id"
    )
    fun loadUserAndBookNames(): LiveData<List<UserBook>>
}

data class UserBook(val userName: String?, val bookName: String?)
```

Multimap return types

- » In the multimap return type approach, you don't need to define any additional data classes.
- » Instead, you define a `multimap` return type for your method based on the map structure that you want and define the relationship between your entities directly in your SQL query.

```
@Query(  
    "SELECT * FROM user" +  
    "JOIN book ON user.id = book.user_id"  
)  
fun loadUserAndBookNames(): Map<User, List<Book>>
```


Create embedded objects

- » You'd like to express an entity or data object as a cohesive whole in your database logic, even if the object contains several fields.
- » In these situations, you can use the [@Embedded](#) annotation to represent an object that you'd like to decompose into its subfields within a table.

```
data class Address(  
    val street: String?,  
    val state: String?,  
    val city: String?,  
    @ColumnInfo(name = "post_code") val postCode: Int  
)  
  
@Entity  
data class User(  
    @PrimaryKey val id: Int,  
    val firstName: String?,  
    @Embedded val address: Address?  
)
```

Define one-to-one relationships

- » A ***one-to-one relationship*** between two entities is a relationship where each instance of the parent entity corresponds to exactly one instance of the child entity, and the reverse is also true.
- » **For example**, each user has only one library, and each library corresponds to exactly one user. Therefore, there is a one-to-one relationship between the **User** entity and the **Library** entity.

Example

```
@Entity
data class User(
    @PrimaryKey val userId: Long,
    val name: String,
    val age: Int
)

@Entity
data class Library(
    @PrimaryKey val libraryId: Long,
    val userOwnerId: Long
)
```

```
data class UserAndLibrary(
    @Embedded val user: User,
    @Relation(
        parentColumn = "userId",
        entityColumn = "userOwnerId"
    )
    val library: Library
)
```

```
@Transaction
@Query("SELECT * FROM User")
fun getUsersAndLibraries(): List<UserAndLibrary>
```

Define one-to-many relationships

- » A **one-to-many relationship** between two entities is a relationship where each instance of the parent entity corresponds to zero or more instances of the child entity, but each instance of the child entity can only correspond to exactly one instance of the parent entity.
- » **For example**, the user organizes their songs into playlists. Each user can create as many playlists as they want, but each playlist is created by exactly one user. Therefore, there is a one-to-many relationship between the **User** entity and **Playlist** entity.

Example

```
@Entity
data class User(
    @PrimaryKey val userId: Long,
    val name: String,
    val age: Int
)

@Entity
data class Playlist(
    @PrimaryKey val playlistId: Long,
    val userCreatorId: Long,
    val playlistName: String
)
```

```
data class UserWithPlaylists(
    @Embedded val user: User,
    @Relation(
        parentColumn = "userId",
        entityColumn = "userCreatorId"
    )
    val playlists: List<Playlist>
)
```

```
@Transaction
@Query("SELECT * FROM User")
fun getUsersWithPlaylists(): List<UserWithPlaylists>
```

Define many-to-many relationships

- » A **many-to-many relationship** between two entities is a relationship where each instance of the parent entity corresponds to zero or more instances of the child entity, and the reverse is also true.
- » **For example**, the songs in the user-defined playlists. Each playlist can include many songs, and each song can be a part of many different playlists. Therefore, there is a many-to-many relationship between the **Playlist** entity and the **Song** entity.

Example

```
@Entity
data class Playlist(
    @PrimaryKey val playlistId: Long,
    val playlistName: String
)

@Entity
data class Song(
    @PrimaryKey val songId: Long,
    val songName: String,
    val artist: String
)

@Entity(primaryKeys = ["playlistId", "songId"])
data class PlaylistSongCrossRef(
    val playlistId: Long,
    val songId: Long
)
```

```
data class PlaylistWithSongs(
    @Embedded val playlist: Playlist,
    @Relation(
        parentColumn = "playlistId",
        entityColumn = "songId",
        associateBy = Junction(PlaylistSongCrossRef::class)
    )
    val songs: List<Song>
)

data class SongWithPlaylists(
    @Embedded val song: Song,
    @Relation(
        parentColumn = "songId",
        entityColumn = "playlistId",
        associateBy = Junction(PlaylistSongCrossRef::class)
    )
    val playlists: List<Playlist>
)
```

```
@Transaction
@Query("SELECT * FROM Playlist")
fun getPlaylistsWithSongs(): List<PlaylistWithSongs>

@Transaction
@Query("SELECT * FROM Song")
fun getSongsWithPlaylists(): List<SongWithPlaylists>
```

Prepopulate your database



Prepopulate from an app asset

- » To prepopulate a Room database from a prepackaged database file that is located anywhere in your app's assets/ directory, call the [`createFromAsset\(\)`](#) method from your RoomDatabase.Builder object before calling [`build\(\)`](#):

```
Room.databaseBuilder(appContext, AppDatabase::class.java, "Sample.db")  
    .createFromAsset("database/myapp.db")  
    .build()
```



Prepopulate from the file system

- » To prepopulate a Room database from a prepackaged database file that is located anywhere in the device's file system, call the [`createFromFile\(\)`](#) method from your `RoomDatabase.Builder` object before calling [`build\(\)`](#):

```
Room.databaseBuilder(appContext, AppDatabase::class.java, "Sample.db")  
    .createFromFile(File("mypath"))  
    .build()
```



Migrate your Room database



Automated migrations

- » To declare an automated migration between two database versions, add an [@AutoMigration](#) annotation to the [autoMigrations](#) property in [@Database](#).

```
// Database class before the version update.
@Database(
    version = 1,
    entities = [User::class]
)
abstract class AppDatabase : RoomDatabase() {
    ...
}

// Database class after the version update.
@Database(
    version = 2,
    entities = [User::class],
    autoMigrations = [
        AutoMigration (from = 1, to = 2)
    ]
)
abstract class AppDatabase : RoomDatabase() {
    ...
}
```

Automatic migration specifications

- » If Room detects ambiguous schema changes and it can't generate a migration plan without more input, it throws a compile-time error and asks you to implement an `AutoMigrationSpec`.
- » Use `AutoMigrationSpec` to give Room the additional information that it needs to correctly generate migration paths.
- » To use the `AutoMigrationSpec` implementation for an automated migration, set the `spec` property in the corresponding `@AutoMigration` annotation.

```
@Database(  
    version = 2,  
    entities = [User::class],  
    autoMigrations = [  
        AutoMigration (  
            from = 1,  
            to = 2,  
            spec = AppDatabase.MyAutoMigration::class  
        )  
    ]  
)  
abstract class AppDatabase : RoomDatabase() {  
    @RenameTable(fromTableName = "User", toTableName = "AppUser")  
    class MyAutoMigration : AutoMigrationSpec  
    ...  
}
```

Manual migrations

- » For example, if you decide to split the data in a table into two tables, Room can't tell how to perform this split. In cases like these, you must manually define a migration path by implementing a [Migration](#) class.

```
val MIGRATION_1_2 = object : Migration(1, 2) {  
    override fun migrate(database: SupportSQLiteDatabase) {  
        database.execSQL("CREATE TABLE `Fruit` (`id` INTEGER, `name` TEXT, "  
            "PRIMARY KEY(`id`))")  
    }  
}  
  
val MIGRATION_2_3 = object : Migration(2, 3) {  
    override fun migrate(database: SupportSQLiteDatabase) {  
        database.execSQL("ALTER TABLE Book ADD COLUMN pub_year INTEGER")  
    }  
}
```

Defines a migration path between a startVersion and an endVersion by overriding the [Migration.migrate\(\)](#) method.

```
Room.databaseBuilder(applicationContext, MyDb::class.java, "database-name")  
    .addMigrations(MIGRATION_1_2, MIGRATION_2_3).build()
```

Add Migration classes to your database builder using the [addMigrations\(\)](#) method.

Test database



Test Database

- » There are 2 ways to test your database:
- On an Android device.
 - On your host development machine (not recommended).



Test on an Android device

- » The recommended approach for testing your database implementation is writing a **JUnit** test that runs on an Android device.
- » Because these tests don't require creating **an activity**, they should be faster to execute than your UI tests.

```
@RunWith(AndroidJUnit4::class)
class SimpleEntityReadWriteTest {
    private lateinit var userDao: UserDao
    private lateinit var db: TestDatabase

    @Before
    fun createDb() {
        val context = ApplicationProvider.getApplicationContext<Context>()
        db = Room.inMemoryDatabaseBuilder(
            context, TestDatabase::class.java).build()
        userDao = db.getUserDao()
    }

    @After
    @Throws(IOException::class)
    fun closeDb() {
        db.close()
    }

    @Test
    @Throws(Exception::class)
    fun writeUserAndReadInList() {
        val user: User = TestUtil.createUser(3).apply {
            setName("george")
        }
        userDao.insert(user)
        val byName = userDao.findUsersByName("george")
        assertEquals(1, byName.size())
    }
}
```

Debug database

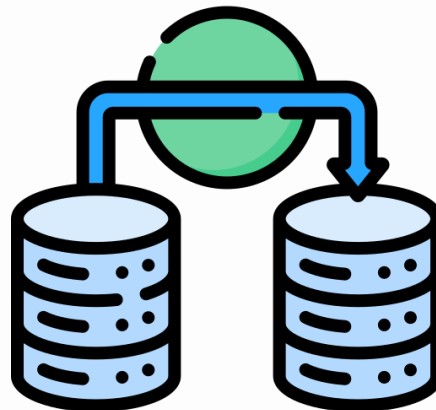
- » There are several tools and processes that you can use to debug your database:
 - » **Use the Database Inspector**
 - » **Dump data from the command line**

Migrate from SQLite to Room



Migrate from SQLite to Room

- » The **Room** persistence library provides a number of benefits over using the **SQLite APIs** directly:
 - Compile-time verification of **SQL** queries
 - Convenience annotations that minimize repetitive and error-prone boilerplate code
 - Streamlined database migration paths
- » If your app currently uses a **non-Room** implementation of SQLite, you can migrate your app to use Room instead.



Migrate steps

- » Update dependencies
- » Update model classes to data entities
- » Create DAOs
- » Create a database class
- » Define a migration path
- » Update the database instantiation
- » Test your implementation



Save data using SQLite



Define a Schema and Contract

- One of the main principles of SQL databases is the schema: a formal declaration of how the database is organized.
- Create a companion class, known as a contract class, which explicitly specifies the layout of your schema in a systematic and self-documenting way.
- A contract class is a container for constants that define names for URIs, tables, and columns. It allows you to use the same constants across all the other classes in the same package.

Define a Schema and Contract (2)

- For example, the following contract defines the table name and column names for a single table representing an RSS feed:

```
object FeedReaderContract {  
  // Table contents are grouped together in an anonymous object.  
  object FeedEntry : BaseColumns {  
    const val TABLE_NAME = "entry"  
    const val COLUMN_NAME_TITLE = "title"  
    const val COLUMN_NAME_SUBTITLE = "subtitle"  
  }  
}
```


Create a database using an SQL helper

- » Implement methods that create and maintain the database and tables. Here are some typical statements that create and delete a table:

```
private const val SQL_CREATE_ENTRIES =  
    "CREATE TABLE ${FeedEntry.TABLE_NAME} (" +  
        "${BaseColumns._ID} INTEGER PRIMARY KEY," +  
        "${FeedEntry.COLUMN_NAME_TITLE} TEXT," +  
        "${FeedEntry.COLUMN_NAME_SUBTITLE} TEXT)"  
  
private const val SQL_DELETE_ENTRIES = "DROP TABLE IF EXISTS ${FeedEntry.TABLE_NAME}"
```

Create a database using an SQL helper (2)

» To use [SQLiteOpenHelper](#), create a subclass that overrides the [onCreate\(\)](#) and [onUpgrade\(\)](#) callback methods.

```
class FeedReaderDbHelper(context: Context) : SQLiteOpenHelper(context, DATABASE_NAME, null, DATABASE_VERSION) {
    override fun onCreate(db: SQLiteDatabase) {
        db.execSQL(SQL_CREATE_ENTRIES)
    }
    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {
        // This database is only a cache for online data, so its upgrade policy is
        // to simply to discard the data and start over
        db.execSQL(SQL_DELETE_ENTRIES)
        onCreate(db)
    }
    override fun onDowngrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {
        onUpgrade(db, oldVersion, newVersion)
    }
    companion object {
        // If you change the database schema, you must increment the database version.
        const val DATABASE_VERSION = 1
        const val DATABASE_NAME = "FeedReader.db"
    }
}
```

the [onDowngrade\(\)](#) and [onOpen\(\)](#) methods are not required.

Put information into a database

- » Insert data into the database by passing a [ContentValues](#) object to the [insert\(\)](#) method:

```
// Gets the data repository in write mode
val db = dbHelper.writableDatabase

// Create a new map of values, where column names are the keys
val values = ContentValues().apply {
    put(FeedEntry.COLUMN_NAME_TITLE, title)
    put(FeedEntry.COLUMN_NAME_SUBTITLE, subtitle)
}

// Insert the new row, returning the primary key value of the new row
val newRowId = db?.insert(FeedEntry.TABLE_NAME, null, values)
```

Read information from a database

- » To read from a database, use the [query\(\)](#) method, passing it your selection criteria and desired columns. The method combines elements of [insert\(\)](#) and [update\(\)](#).

```
val db = dbHelper.readableDatabase

// Define a projection that specifies which columns from the database
// you will actually use after this query.
val projection = arrayOf(BaseColumns._ID, FeedEntry.COLUMN_NAME_TITLE, FeedEntry.COLUMN_NAME_SUBTITLE)

// Filter results WHERE "title" = 'My Title'
val selection = "${FeedEntry.COLUMN_NAME_TITLE} = ?"
val selectionArgs = arrayOf("My Title")

// How you want the results sorted in the resulting Cursor
val sortOrder = "${FeedEntry.COLUMN_NAME_SUBTITLE} DESC"

val cursor = db.query(
    FeedEntry.TABLE_NAME,    // The table to query
    projection,              // The array of columns to return (pass null to get all)
    selection,               // The columns for the WHERE clause
    selectionArgs,           // The values for the WHERE clause
    null,                   // don't group the rows
    null,                   // don't filter by row groups
    sortOrder                // The sort order
)
```

The results of the query are returned to you in a [Cursor](#) object.

Delete information from a database

- » To delete rows from a table, you need to provide selection criteria that identify the rows to the [delete\(\)](#) method.

```
// Define 'where' part of query.  
val selection = "${FeedEntry.COLUMN_NAME_TITLE} LIKE ?"  
// Specify arguments in placeholder order.  
val selectionArgs = arrayOf("MyTitle")  
// Issue SQL statement.  
val deletedRows = db.delete(FeedEntry.TABLE_NAME, selection, selectionArgs)
```



The return value for the [delete\(\)](#) method indicates the number of rows that were deleted from the database.

Update a database

- » When you need to modify a subset of your database values, use the [update\(\)](#) method.

```
val db = dbHelper.writableDatabase

// New value for one column
val title = "MyNewTitle"
val values = ContentValues().apply {
    put(FeedEntry.COLUMN_NAME_TITLE, title)
}

// Which row to update, based on the title
val selection = "${FeedEntry.COLUMN_NAME_TITLE} LIKE ?"
val selectionArgs = arrayOf("MyOldTitle")
val count = db.update(
    FeedEntry.TABLE_NAME,
    values,
    selection,
    selectionArgs)
```



The return value of the [update\(\)](#) method is the number of rows affected in the database.

Persisting database connection

- » Since [`getWritableDatabase\(\)`](#) and [`getReadableDatabase\(\)`](#) are expensive to call when the database is closed, you should leave your database connection open for as long as you possibly need to access it.
- » Typically, it is optimal to close the database in the [`onDestroy\(\)`](#) of the calling Activity.

```
override fun onDestroy() {  
    dbHelper.close()  
    super.onDestroy()  
}
```



Thank You !

OPEN MIND - OPEN DOORS